CEREX / LOCAL AGENTIC INFRASTRUCTURE

Your PC becomes a coordinated agent workforce.

Separate Core Engine - Architecture, PC Diagnostics, Workspace Scheduling and LLM Communication

CereX is not merely a remote desktop for an external AI. CereX should interpret a mission, separate it into independently scheduled jobs, coordinate local workers, communicate with one or more LLMs, and combine results without occupying the user's active desktop.

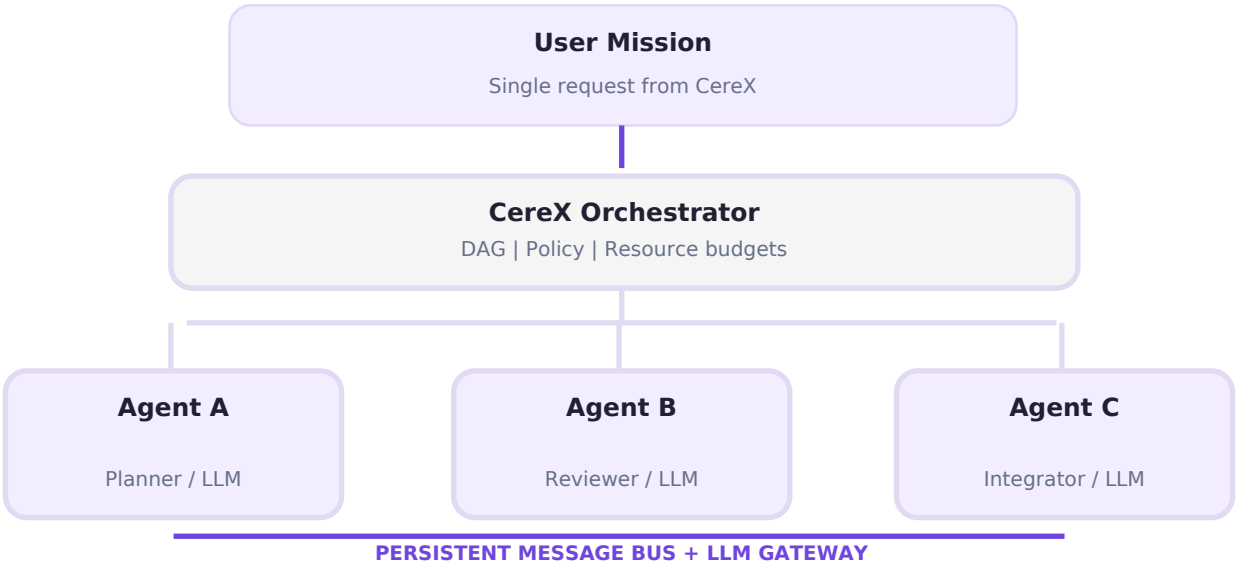
THIS DELIVERY	REALITY CHECK
Repository-ready source package	Working pre-alpha prototype, not production-ready CereX integration
PC optimization	Read-only resource assessment + worker throttling; no registry or process manipulation
Desktop separation	Headless background OS processes only; GUI virtualization is not implemented
Two-way communication	SQLite mailbox + multi-round agent/LLM exchange proven in offline tests

Design rule: CereX orchestrates, LLMs reason, local workers execute within explicitly granted boundaries.

Desktop focus, company credentials, user files, and active sessions remain outside the default execution scope. This package does not include the previous CereX Web UAT binaries; it is intended to be developed alongside them.

1 / Platform architecture

One mission splits into jobs, each with its own headless process and job workspace. CereX schedules job dependencies and the resource-aware concurrency budget.



Layer	Responsibility
PC Intelligence	CPU, RAM, storage, battery, load and conservative worker recommendation
DAG Orchestrator	Validation, dependencies, low-priority workers, timeouts and result aggregation
Message Bus	Durable local messages with recipient, sender and explicit acknowledgement
LLM Gateway	Explicit opt-in OpenAI-compatible endpoints; model routing by job
CereX Web integration	Proposed authenticated control-plane API (not currently wired)

Customizable parameters

The mission JSON contains max_workers, role prompts, dependencies, timeouts, model/provider settings, and thresholds for CPU, RAM, disk and battery. Network LLM execution is disabled unless explicitly requested.

2 / PC diagnosis and optimization

The local doctor reads resource indicators and recommends a safe workload without modifying Windows settings or interrupting applications.

Signals	Current implementation
CPU	Logical cores, usage percentage when psutil is available
Memory	Total/available RAM and memory pressure
Storage	Free disk space, disk utilization
Battery	Percentage and charging status when reported by OS
Safety policy	Concurrency based on configurable CPU/RAM/storage/battery limits

How adaptive execution works

If the machine is under heavy CPU or memory load, low on free disk, or running on critically low battery, new jobs are deferred. If the PC becomes moderately busy, the scheduler reduces how many fresh workers may start. Running tasks are not yet dynamically paused or checkpointed.

Current Windows status

At the time of this delivery, the paired Windows devices were reported OFFLINE by the remote controller. Consequently, no current live diagnostic result for your Windows PC is claimed. Use DIAGNOSE_PC_WINDOWS.bat on the target computer or reconnect it for an authorized diagnosis.

Sample command

```
py -3 -m cerex_core doctor --max-workers 4
```

3 / Active desktop stays usable

The goal is separate computer work, not remote control of the user's current screen.

Execution type	Now	Limits
Headless Python workers	Available	Separate background OS processes + isolated output folders, not security sandboxes
Per-worker resource policy	Partial	Below-normal priority on Windows; conservative startup; no hard CPU/RAM quotas
Code execution sandbox	Not implemented	Requires Docker or VM, tool permissions and audit
Independent GUI workspace	Not implemented	Requires real separate VM/session, not hidden clicks on active desktop

Important isolation distinction

Background process means no intended UI interaction. Virtual desktop/session means independent GUI focus, display and input. Security sandbox means restricted access to host resources. They are three different guarantees and must never be marketed as interchangeable.

Security defaults

LLM responses are data. Worker tools do not permit direct OS shell execution, interactive browser actions, host keyboard input, registry modification, or access to the primary desktop. A local API is bound to 127.0.0.1 and requires a bearer token.

4 / Agent-to-agent and LLM feedback

The prototype implements a real local mailbox and a multi-round loop, not merely a one-way assignment.

Step	Offline test execution
1	Architect sends a review_request event to Reviewer
2	Reviewer receives it from SQLite and sends review_response
3	Architect reads that response and performs another model reasoning round
4	Integrator consolidates final outputs after dependencies complete

Supported model pattern

A worker sends mission context, job instructions, prior results and inbox messages to an OpenAI-compatible chat-completions endpoint. The model can return structured peer messages and request another LLM round after feedback. Offline mock responses make this testable without spending tokens.

LLM routing

The base URL, model and API-key environment-variable name are configurable globally or per job. Only HTTPS providers and loopback HTTP are permitted. Native Anthropic/Google SDK transport adapters, tool streaming, billing metering, MCP/A2A and enterprise DLP are not yet implemented.

No automatic model spending

Default mode is mock, and the real model is opt-in with --live. Production must add per-mission and per-provider hard budgets before unattended operation.

5 / Setup, integration and validation

Open the ZIP as a separate folder or Git repository, then start with read-only diagnosis and the offline demo.

Windows quick start

1. Install Python 3.10+ and optionally psutil:

```
py -3 -m pip install psutil
```

2. Diagnose this PC:

```
py -3 -m cerex_core doctor --max-workers 4
```

3. Run two-way agent communication without external requests:

```
py -3 -m cerex_core run --config configs/mission_interagent_demo.json
```

4. Run tests:

```
py -3 -m unittest discover -s tests -v
```

Local bridge to CereX Web (experimental)

The prototype includes a token-protected loopback API: /health, /diagnostics, POST /missions and GET /missions/{id}. Launching the server requires CEREX_CORE_TOKEN. Do not expose it through a public reverse proxy. Remote device authentication and the current CereX cloud integration are still planned.

Validation	Result
Automated offline tests	12 passing tests in the build environment
Agent request-response-request	Verified using the message bus, peer feedback and next-round processing
Authenticated local HTTP endpoints	Verified with unauthorized rejection + authorized mission completion
Live Windows and GUI automation	Not verified; Windows paired devices offline

Next milestone

Build a truly isolated GUI backend (VM or independent session), local-device auth channel, scoped tool permissions, durable resume/checkpoints, retries, budget caps, and validation on the actual Windows PC. Integrate with existing CereX Web through a signed control channel rather than replacing the production UAT.

Related open-source references: OpenHands, Microsoft Agent Framework, LangGraph and psutil. See docs/OPEN_SOURCE_REFERENCES.md in the ZIP.